Lab 4: Forward and Inverse Kinematics

Varun Senthil Kumar, vsenth13@asu.edu, ASU ID: 1233679566

***Abstract*— This lab focuses on analyzing and implementing the forward and inverse kinematics of the Dobot Magician Lite robotic arm. The objectives were to gain practical experience in deriving and applying kinematic equations, constructing homogeneous transformation matrices, and validating joint-to-end-effector relationships through simulation and Python programming. Using the pydobot library, the robot was programmed to perform point-to-point motion tasks where both forward kinematics (end-effector pose from joint angles) and inverse kinematics (joint angles from target positions) were tested. The lab provided hands-on insight into robotic motion control, coordinate transformations, and the mathematical foundations of manipulator kinematics.**

I. Problem Statement

Precise positioning and trajectory planning are fundamental requirements in robotic manipulation tasks, which depend on understanding the mathematical relationship between a robot's joint space and its Cartesian workspace. For the Dobot Magician Lite, this involves implementing forward kinematics to calculate the end-effector pose from known joint angles, and inverse kinematics to determine the required joint variables for a specified target position. The challenge lies in accurately formulating and validating these kinematic models to ensure reliable point-to-point motion.

This lab addresses these challenges by deriving and testing the forward and inverse kinematics of the Dobot Magician Lite, enabling the robot to perform controlled pick-and-place operations and providing practical insight into the connection between theoretical kinematic models and physical robotic execution.

II. Procedure

All required packages and dependencies were installed on the Linux machine. After the hand calculations for the homogeneous transformation matrices were done, it was implemented on MATLAB. Then, using the DobotLab online simulator, the values of both forward and inverse kinematics were verified.

Once the setup was complete, based on the MATLAB code, a python script was developed to calculate the homogeneous transformations, forward and inverse kinematics for the given coordinates and joints respectively. The Dobot Magician Lite was connected, and the python script was run.

For forward kinematics, the python script calculates the homogeneous transformation matrices and gets the coordinates that the Dobot is supposed to be at. Then, the joint values are passed to the Dobot, and the Dobot's final position is verified against the value calculated from the homogeneous transformation. Table 1 shows the joint values and returned coordinates.

For inverse kinematics, the python script calculates the joints of the Dobot at specified coordinates. Then, the coordinates are passed to the Dobot, and the joint values of the Dobot at that position is verified against the input joint values. Table 2 shows the coordinates and the returned joint values.

Table 1: Forward Kinematics

	x	y	z
Theta(1-4) = [0,0,0,0]	240	0	150
Theta(1-4) = [90,20,30,0]	0	272.5	68.54
Theta(1-4) = [-2,25,23,45]	291.29	-10.17	77.34
Theta(1-4) = [31,52,42,0]	274.01	164.64	-8.02
Theta(1-4) = [27,5224,0]	301.61	156.73	31.34

Table 2: Inverse Kinematics

	Theta 1	Theta 2	Theta 3
(x,y,z) = (240,0,150)	0	0	0
(x,y,z) = (0,272.5,68.54)	90	20	30
(x,y,z) = (300,50,100)	9.46	26.95	12.99
(x,y,z) = (280,-195,15)	-34.85	53.60	29.56
(x,y,z) = (35,270,-33)	82.61	48.39	62.14

III. Results

The Dobot successfully executed the forward and inverse kinematics. The positions and joints were calculated with an accuracy of ±5mm/degrees. The physical position of the Dobot was also verified against the DobotLab simulator positions.

Overall, the Dobot was able to complete the intended operations, validating the programmed motion sequences and highlighting the importance of forward and inverse kniematics in robotic movement.

IV. Discussion

The base frame's orientation directly influences the sign of the z coordinate in forward kinematics. If the base frame's z-axis is off, then it means that all the vertical coordinates in the forward kinematics will also be off.

In forward kinematics, matrix multiplication order is critical because each homogeneous transform is defined relative to a specific frame. Since matrix multiplication is not commutative, changing the order produces a completely different pose.

Generally, inverse kinematics may give multiple solutions. At the workspace boundary, since the geometry is constrained, the inverse kinematics solutions reduces to a single solution. The robot is at or near a singularity, causing unstable solutions.

In certain cases, the inverse kinematics may yield no solutions. Such examples are when the target is outside the reachable workspace, inside the unreachable workspace due to numerical constraints. The inverse kinematic failures should be handled explicitly, and the program should not allow the Dobot to extend beyond its limitations, as it may damage the Dobot.

V. Technical specifications

Robot Name: Dobot Magician Lite
Robot Serial Number: DT-MGL-4R002-01E
Repeatability: ±0.2mm
Maximum Payload: 0.25kg
Maximum Reach: 340mm
End Effector: Suction Cup End Effector
Software Platform: Python
Safety Features: Collision Detection

VI. Code

Repository Link:
https://github.com/VarunPro23/RAS/tree/main/Lab%204

Repository content:

- *Lab4.py* – code for Lab 4
- *ht*.py – code for homogeneous matrices.
- *Fk*-ik.m – MATLAB Code